先看整个项目，再理解今天为什么写这些代码

我们正在做一个“登录后管理自己学习任务”的系统。你看到的网页、Java 代码和数据库，都是同一个任务流程的不同部分。

> **今天的位置：**你已能创建和查询任务。今天让已有任务可以修改、删除，并保证别人不能操作你的记录。先看全貌，再进入代码，最后亲手写一个小片段。

阅读顺序：先读第 1～4 节建立地图，再读第 5 节追踪一个具体动作。理解动作后再读第 6～7 节代码。一次读到当前步骤即可，不要求背下整份文档。

图片均为教学示意图，说明真实项目的关系；不是运行页面的截图。文件可离线阅读，也可在浏览器按 Ctrl+P 打印或另存 PDF。

1. 我们为什么要写这个系统?

先想一个实际场景：你有“复习 Java” “练习 Vue”两条任务。今天记录进度，明天打开电脑，还想看到这些任务；换一个账号，则应该看到另一个人的学习记录。

一个完整任务至少有三个阶段：**记录下来** → **跟踪进度** → **查看结果**。因此我们需要创建、查询、修改、删除和统计。这些操作统称增删改查，常写作 CRUD。

你想做什么	网页怎么做	后端怎么接收	数据库做什么
查看自己的任务	打开任务管理页	GET /api/tasks	SELECT：读取记录
添加学习计划	新建弹窗填写并创建	POST /api/tasks	INSERT：新增一行
修改内容或进度	编辑弹窗或状态下拉框	PUT /api/tasks/任务编号	UPDATE：改变已有行
移除不需要的计划	删除并确认	DELETE /api/tasks/任务编号	DELETE：删除指定行
查看完成情况	打开学习统计	GET /api/tasks/stats	COUNT、SUM：按当前用户计算数量

为什么只改网页数组不够?

网页中的 `tasks` 数组可以临时显示任务。但刷新网页会重新运行页面代码，临时数组不能代替持久保存。MySQL 负责长期保存，网页每次加载时从后端重新查询记录。

同样，只在网页上把状态显示为“已完成”也不够：数据库不变，重新读取时仍是“未开始”。今天写 UPDATE，就是让这次进度变化真正保存下来。

> 每一段代码都服务于一个用户动作。学 SQL 时先想“我希望哪条记录发生什么变化”，再决定使用 SELECT、INSERT、UPDATE 还是 DELETE。

2. 整个系统由什么组成?

现在前端和后端都在你的电脑上，数据库也在本机。“前端”和“后端”表示职责不同，并不要求两台电脑。



图 1：项目全貌。只有一条业务链，不是五个互不相关的小项目。

名称	在本项目里的职责	具体文件
Vue + Element Plus	页面数据、按钮事件、表格与弹窗	前端 src/App.vue
Controller	识别接口地址、接收参数、返回结果	TaskController.java
Service	从 token 确认用户，组织任务操作	TaskService.java
Mapper + MyBatis	把 Java 方法和 XML SQL 对应起来	TaskMapper.java、TaskMapper.xml
MySQL	保存账号与任务，执行查询和修改	learning_app 中的 users、tasks 表

不要把 Spring Boot、Controller 和 Service 看成三种互相独立的后端。Controller、Service 都是这个 Spring Boot 项目里的 Java 类。

3. 前面的课程怎样组成现在的网页?

下面按桌面 Demo 名称和我们已完成的联调整理学习路线。它说明每个技术解决什么问题，不代表每个旧 Demo 都已经合并进当前项目。

学习阶段	新增的能力	你会看到什么变化
01 HTML、02 CSS	定义页面内容、布局与样式	能显示标题、输入框、按钮；页面先有外形
03 JavaScript、04 Fetch	处理点击、改变数组、发送网络请求	按钮开始有动作；能从接口获取内容
05 Vue、06 Router、07 Pinia、08 Axios	组件与响应式、路由、共享状态、请求工具	数据变化能更新页面；学习了多种组织代码方式
09 Element Plus	使用表格、弹窗、表单、消息组件	形成当前“学习空间”的界面基础
10 Spring Boot	用 Java 提供接口，先练内存 CRUD	页面可以请求自己的后端；内存保存仍受进程生命周期影响
11 MySQL、12 JPA、13 MyBatis	建立数据库、学习两种数据库访问方式	记录可以长期保存；当前项目采用 MyBatis
14 JWT、前后端登录联调	注册、登录、token、当前用户	先登录，再进入任务管理；账号能够区分
17 ~ 18 任务持久化与新增	任务表、查询、创建、输入校验	任务不再只是固定数组，刷新和重新登录能保留
今天：修改、删除与归属	UPDATE、DELETE、id 与 user_id 条件	进度和编辑结果能保存；只允许操作自己的记录

> **页面外观相似，不代表没有变化。**从前端数组改成 MySQL 保存，表格可能完全一样，但刷新后的行为、账号之间的数据归属、接口失败时的提示都变了。

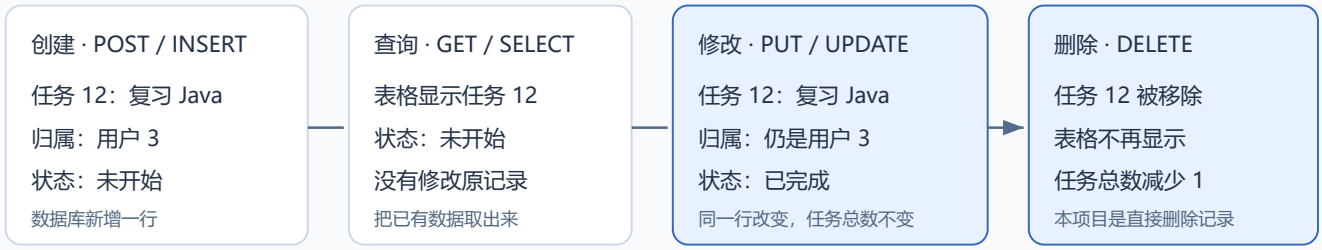
当前项目的实际状态

我们已经提前写好了任务增删改查、统计和筛选，页面基本操作已由你验证。今天主要是把已有功能的设计和书写过程补清楚。网页已经能用，不能作为“今天的代码已经学会”的依据。

当前使用原生 `fetch` 请求，三个菜单主要通过 `activePage` 切换内容；Vue Router 正式接入、组件拆分和生产部署是后续任务。之前学过 Router、Pinia、Axios，并不代表当前项目已经使用了它们。

4. 今天要让网页和数据发生什么变化?

用同一条任务串起来。下面编号仅为示例，不是让你直接修改数据库中的任务 12。



今天聚焦右侧两步：UPDATE 改变已有任务，DELETE 移除指定任务。

每一步都先确认用户身份；后端拒绝操作他人任务。这里显示的是教学流程，不是实际按钮。

图 2：内容会改变，任务身份与用户归属在编辑时保持不变。

修改的验收

编辑名称、分类或优先级；改变状态；刷新后仍保留。任务总数不变，完成数量随状态变化。

删除的验收

确认删除后记录消失，刷新不会回来，统计同步减少。别人不能靠改任务编号删掉你的记录。

取消删除弹窗时，没有发送删除请求。确认以后，后端成功删除，页面才显示成功。本项目直接移除数据库记录，没有回收站功能。

5. 从“改为已完成”开始，走完一次请求

这里把任务 12 改为“已完成”，假设登录用户编号是 3。与其记五个孤立文件，不如跟着这一个动作走。



图 3：同一次请求贯穿所有层。你学的 SQL 位于第 ④ 步；setStatus 位于第 ③ 步。

为什么要传这些信息？

信息	解决的问题	从哪里得到
任务编号 id	我要改哪条任务？	已查询到的任务对象，放入地址
新字段值	我要把它改成什么？	网页输入，放入 JSON 请求体
token	请求者是谁？	登录成功后保存的凭证，放入请求头
用户编号 userId	这条任务是否属于请求者？	后端验证 token 后取得，前端不能指定

`JSON` 是请求和响应的数据格式；`TaskRequest` 是接收内容的 Java 对象；`Task` 是包含编号与归属、供数据库操作使用的对象。它们是同一段数据在不同步骤的表现形式。

6. 今天的代码写在哪里，为什么这样写?

前端目录：`C:/Users/李佳恒/Desktop/09-ElementPlus-后台Demo`。后端目录：`C:/Users/李佳恒/Desktop/14-SpringBoot-JWT-Demo`。行号为 2026-10-14 当前文件位置，后续编辑可能变化。

动作	前端 App.vue	Controller	Service	Mapper / XML
编辑弹窗	203 打开；209 保存，编辑分支发 PUT	TaskController.java 28	TaskService.java 36	TaskMapper.java 13；XML 29
直接改状态	232 updateTask	同上：update	同上：update	同上：update
删除	255 removeTask	TaskController.java 34	TaskService.java 49	TaskMapper.java 14；XML 35

后端 Java 文件都在 `src/main/java/com/learning/jwtapi/`；XML 在 `src/main/resources/mapper/TaskMapper.xml`。在编辑器按 Ctrl+G 输入行号定位。讲义阅读时先看下面摘录，不需要来回打开五个窗口。

6.1 前端：把用户动作变成请求

```
// 状态变化时的请求摘录
await apiRequest(`/api/tasks/${task.id}`, {
    method: "PUT",
    body: JSON.stringify({
        title: task.title,
        category: task.category,
        priority: task.priority,
        status: task.status,
    }),
});
```

这里传完整可编辑字段，因为当前 PUT 接口会更新这些字段。只发送 status 的片段适合教学说明，不能直接替换正式请求；否则其他必填字段缺失会导致 400。请求头由 `apiRequest` 加入 token。

6.2 Controller：读取地址、内容和凭证

```
@PutMapping("/{id}")
public TaskResponse update(
    @PathVariable Long id,
    @Valid @RequestBody TaskRequest request,
    @RequestHeader(value = "Authorization", required = false)
    String authorization) {
    return service.update(id, request, authorization);
}
```

`@PathVariable` 从地址取任务编号；`@RequestBody` 从 JSON 取新内容；`@Valid` 触发输入规则；`@RequestHeader` 取凭证。允许请求头为空是为了由 Service 统一返回 401，不代表允许匿名修改。

6.3 Service：确认用户，组装要修改的数据

```
public TaskResponse update(Long id, TaskRequest request,
                           String authorization) {
    Long userId = currentUserId(authorization);
    Task task = new Task();
    task.setId(id);
    task.setUserId(userId);
    task.setTitle(request.title().trim());
    task.setCategory(request.category().trim());
    task.setPriority(request.priority().trim());
    task.setStatus(request.status().trim());
    if (mapper.update(task) == 0) throw notFound();
    return TaskResponse.from(requireOwned(id, userId));
}
```

你已经练过创建对象和 set 方法。修改比新增多了 `task.setId(id)`，因为新增由数据库生成编号，修改需要指定已有记录。`mapper.update` 返回受影响行数；本项目配置下，没有匹配到可修改任务时返回 0，Service 返回 404。更新后再查询并转换为给前端的响应对象。

6.4 Mapper 与 SQL：Java 方法对应数据库操作

```
// TaskMapper.java
int update(Task task);

// TaskMapper.xml 对应的 SQL
UPDATE tasks
SET title = #{title}, category = #{category},
    priority = #{priority}, status = #{status}
WHERE id = #{id} AND user_id = #{userId}
```

MyBatis 根据 Mapper 命名空间和 XML 的 `id="update"` 找到 SQL，读取 Task 的属性并绑定参数。`#{...}` 是参数绑定占位符，不能去掉花括号当普通 SQL 名称使用。

数据库中的 `user_id` 与 Java 属性 `userId` 名字不同，但表示同一个任务归属。SET 指定“改什么”，WHERE 指定“允许改哪条”。

7. 为什么修改和删除都要检查归属?

前端表格只显示自己的任务，但请求地址可以被人为修改。服务器不能因为“按钮在网页里”就相信请求一定合法。

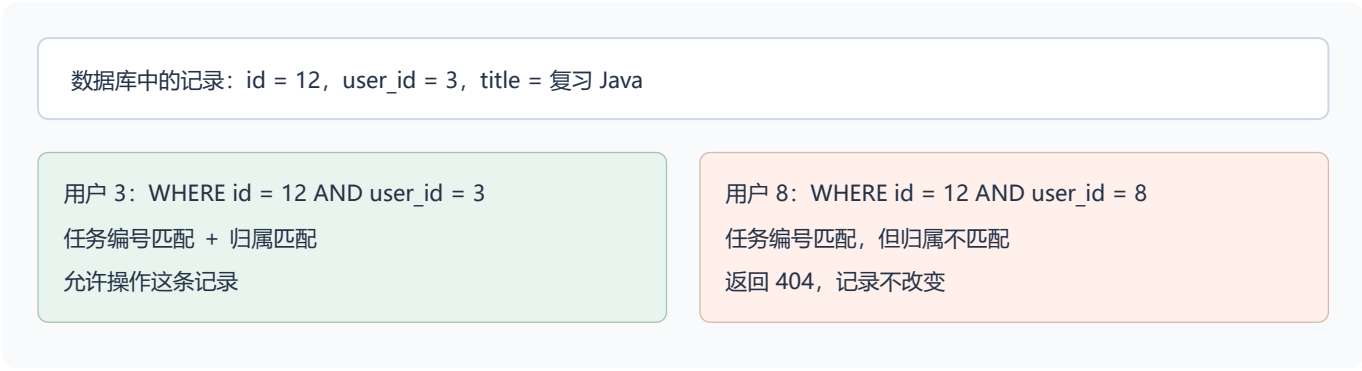


图 4：任务编号确定目标，用户编号确定权限；AND 表示两个条件同时成立。

删除沿用相同的归属规则

```
// Service：身份从 token 得到
public void delete(Long id, String authorization) {
    Long userId = currentUserId(authorization);
    if (mapper.delete(id, userId) == 0) throw notFound();
}

// SQL：只删除属于当前用户的指定任务
DELETE FROM tasks
WHERE id = #{id} AND user_id = #{userId}
```

删除不需要新的 title 或 status，因此不必组装完整 TaskRequest。它只需要“删哪条”和“当前是谁”。前端取消确认时不发请求，确认后发送 DELETE，成功返回 204，没有 JSON 响应体。

接口对“不存在的任务”和“属于别人的任务”统一返回 404，避免透露其他用户的任务是否存在。缺少或失效 token 则返回 401；这两个错误原因不同。

8. 今天怎样从看懂走到会写?

下面先提供完整例子，再安排相近练习。草稿片段不代表可以粘进正式类中直接编译，也不要把教学 SQL 直接对现有数据库执行。真正整合项目时，我会先说明类、方法、参数和文件位置。

第一步：自己描述要改变的记录

> 参考描述：“我要把自己的一条已有任务改成已完成。任务编号不变，所属用户不变，只改变状态。刷新后仍是已完成。”

第二步：跟写最小 SQL

```
UPDATE tasks
SET status = #{status}
WHERE id = #{id} AND user_id = #{userId}
```

在草稿编辑器中自己输入。只关注三个意思：修改任务表、指定新状态、同时限制编号与归属。第一遍可以照写，不要求凭空记住标点。

第三步：写相近变化

把上面的“改状态”变成“改优先级”，保留归属条件。可以先参照示例，写完再展开参考答案：

▶ 参考答案：修改优先级

第四步：把参数来源接到 Java

```
// 仅为草稿：前面的 id、userId、request 已取得
Task task = new Task();
task.setId(id);
task.setUserId(userId);
task.setPriority(request.priority().trim());
```

这几句只准备对象，尚未保存；正式 Mapper 的 update 会读取所有可编辑字段，因此正式调用前需要补齐 title、category、priority、status。你写每一句时都能在图 3 中定位：目前处于 Service 组装对象步骤。

第五步：在项目里验证完整功能

跟写和相近练习完成后，再改网页中的一条自己的练习任务，刷新核对，最后查看统计。已有网页验收通过不需要每天反复点同样按钮；当你实际改了代码时再检查对应行为。

> 今天的完成标准：能把“用户动作 → 参数 → SQL → 数据变化”连起来，并在参考帮助下写出一个相近修改片段。功能是否能用和你是否会写，分别记录。

9. 以后每天先看地图，再进入代码

每天开课前先保存当天讲义。第一部分固定回答：昨天停在哪里、今天解决哪个用户需求、网页会怎样变化、数据库会怎样变化、涉及哪些文件。然后只沿一条完整功能链学习。

环节	建议时间	具体安排
整体地图	20 分钟	需求、页面前后变化、系统关系图；定位今天的部分
概念和设计	25 分钟	为什么传这些参数，为什么用这些接口和 SQL
实际代码追踪	25 分钟	从一个按钮跟到数据库，配准确文件和行号
跟写	55 分钟	从最小片段到方法，逐步增加内容并检查
相近小练习	25 分钟	改变一个字段或规则，必要时提供提示和参考答案
运行验证	15 分钟	针对自己写的内容检查页面、接口和数据
复盘	15 分钟	记录能理解、能跟写、还需帮助的部分

合计约 180 分钟。时间是安排，不靠等待凑时长；卡在概念时先解释清楚，必要时缩减当天功能范围。前一版时间段合计 200 分钟，这里已统一为约三小时。

后续网页还会怎么变?

10/15 整理错误反馈：输入有误、未登录、无权限目标和服务异常，各有明确响应。之后逐日补清前端加载/保存/编辑、统计与筛选的代码，再做会话失效、独立页面路由、组件整理、测试和部署。已提前实现的功能以代码理解、跟写与相近修改为主，不再把点几下网页算成完整学习任务。

后续路由让登录、任务、统计、设置拥有明确地址；加载状态让等待和失败可见；配置与部署让项目可以按指南重新运行。界面未必每节都换样式，但功能、数据可靠性和代码组织会逐步完善。

今天先从这里继续

先读第 1 ~ 4 节，知道当前项目全貌和今天的位置。之后我们从第 5 节的“改为已完成”走完整条链，再进入第 8 节跟写。今天不是让你继续猜几行代码，而是先建立完整功能的上下文。